



01 · Gentle-AI para dummies

Qué es y qué hace cada carpeta

Verificado contra el código · diagramas incluidos

Gentle-AI explicado para dummies

Guía simple de qué es este proyecto, para qué sirve y qué hace cada carpeta. Si nunca tocaste este repo, empezá por acá.

1. ¿Qué es esto en una frase?

Gentle-AI NO es un agente de IA. Es el que le pone los muebles al agente que ya tenés.

Pensalo así: vos ya tenés una casa vacía (Claude Code, Cursor, OpenCode, etc.). Gentle-AI es el equipo que entra, mide, y te instala:

- **Memoria** (que el agente se acuerde de lo que hicieron ayer) → Engram
- **Habilidades** (skills: recetas para tareas comunes)
- **Un método de trabajo** (SDD: pensar antes de codear)
- **Herramientas externas** (MCP servers, CodeGraph, etc.)
- **Una personalidad** que enseña en vez de solo tirar código
- **Un sistema de revisión** con "recibos" para que nada se te escape

Antes: "Instalé Claude Code pero es un chatbot que escribe código." **Después:** "Mi agente tiene memoria, skills, método y me enseña."

2. ¿Por qué existe?

Porque instalar el agente es lo fácil. Lo difícil es **configurarlo bien** para que sea una herramienta seria y no un juguete. Gentle-AI automatiza toda esa configuración para +15 agentes distintos, sin que vos tengas que editar 20 archivos JSON a mano.

Es un **CLI en Go** (se ejecuta en la terminal) con una **interfaz visual** (TUI, esas pantallas coloridas dentro de la terminal).

3. La idea central: dos ejes que NO se mezclan

Esta es LA regla mental más importante del proyecto. Hay dos tipos de conocimiento:

EJE	PREGUNTA QUE RESPONDE	DÓNDE VIVE
Adapters (adaptadores)	"¿DÓNDE va la config de ESTE agente?"	<code>internal/agents/<agente>/</code>
Components (componentes)	"¿QUÉ contenido inyector, para todos los agentes?"	<code>internal/components/<cosa>/</code>

Analogía: el *adapter* es el plano eléctrico de cada casa (sabe dónde está el tablero). El *component* es el electrodoméstico (la heladera funciona igual en cualquier casa). Uno sabe *dónde*, el otro sabe *qué*. Nunca se mezclan.

4. El flujo completo (qué llama a qué)

Cuando corrés `gentle-ai install`, pasa esto, en orden:

```
main.go      → arranca el programa
└─ app/      → decide qué comando pediste (install? sync? doctor?)
  └─ cli/     → lee las flags (--agent, --preset, etc.)
    └─ system/ → detecta tu SO y qué tenés instalado
      └─ catalog/ → consulta qué agentes/componentes existen
        └─ planner/ → ordena las dependencias (¿qué va primero?)
          └─ pipeline/ → ejecuta paso por paso (con rollback si algo falla)
            └─ components/ → inyecta cada cosa (memoria, skills, etc.)
              └─ agents/ → usa el adapter para saber DÓNDE escribir
                └─ verify/ → chequea que todo quedó bien y te da un reporte
```

Precondición (lo que tiene que pasar ANTES): tenés que tener el binario instalado y al menos un agente para configurar.

Postcondición (lo que queda DESPUÉS): tu agente tiene los archivos de config escritos, un backup de lo anterior guardado, y un reporte de qué cambió.

Regla de oro: antes de tocar nada, se hace un **backup**. Siempre hay camino de vuelta.

5. Qué hace cada carpeta (el mapa del repo)

La puerta de entrada

CARPETA	QUÉ HACE	ANALOGÍA
cmd/gentle-ai/	El <code>main.go</code> . Arranca todo y pasa la versión.	La puerta de calle.
internal/app/	Recibe el comando y decide a dónde mandarlo.	La recepción del edificio.

El cerebro que decide

CARPETA	QUÉ HACE	ANALOGÍA
internal/cli/	Modo no-interactivo: lee flags, orquesta install/sync/uninstall/restore.	El formulario que llenás.
internal/tui/	Modo interactivo: las pantallas visuales (Bubbletea, tema Rose Pine).	El asistente con botones.
internal/model/	Los tipos compartidos: IDs de agentes, enums, structs comunes.	El diccionario de términos.
internal/catalog/	La lista de qué agentes y componentes están soportados.	El menú del restaurante.
internal/system/	Detecta tu SO (Mac/Linux/Windows) y qué dependencias tenés.	El inspector de obra.

El planificador y ejecutor

CARPETA	QUÉ HACE	ANALOGÍA
internal/planner/	Resuelve dependencias y las ordena (grafo topológico).	El arquitecto que ordena las tareas.
internal/pipeline/	Ejecuta cada etapa con progreso y rollback si algo falla.	El capataz que ejecuta la obra.
internal/backup/	Saca fotos de la config antes de cambiarla, y sabe restaurar.	El seguro contra incendios.

🔧 Los componentes (el QUÉ)

CARPETA	QUÉ HACE
<code>internal/components/engram/</code>	Instala y configura la memoria (Engram) vía MCP.
<code>internal/components/sdd/</code>	Inyecta el método Spec-Driven Development (prompts y comandos).
<code>internal/components/skills/</code>	Copia las skills (recetas curadas).
<code>internal/components/mcp/</code>	Configura servers MCP como Context7 (docs en vivo).
<code>internal/components/persona/</code>	La personalidad que enseña (Gentleman/Neutral).
<code>internal/components/communitytool/</code>	Instala herramientas de comunidad, ej: CodeGraph.
<code>internal/components/opencodeplugin/</code>	Registra plugins visuales de OpenCode.
<code>internal/components/uninstall/</code>	Limpia prolijamente lo que se instaló.
<code>internal/components/filemerge/</code>	Fusiona archivos con marcadores, sin pisar lo tuyo.

🔧 Los adaptadores (el DÓNDE)

CARPETA	QUÉ HACE
<code>internal/agents/claude/</code>	Sabe dónde van los archivos de Claude Code.
<code>internal/agents/opencode/</code>	Ídem para OpenCode.
<code>internal/agents/cursor/</code> , <code>gemini/</code> , <code>vscode/</code> , <code>codex/</code> , <code>windsurf/</code> , <code>antigravity/</code> ...	Uno por cada agente soportado. Cada uno conoce sus rutas y estrategias.

📦 Assets, estado y mantenimiento

CARPETA	QUÉ HACE	ANALOGÍA
<code>internal/assets/</code>	Los archivos embebidos: prompts, skills, personas, templates.	El depósito de materiales.
<code>internal/skillregistry/</code>	Escanea skills y arma el índice <code>.atl/skill-registry.md</code> .	El bibliotecario.
<code>internal/state/</code>	Guarda qué instalaste en <code>~/.gentle-ai/state.json</code> .	La libreta de "qué le puse a esta casa".
<code>internal/update/</code>	Auto-actualización del binario y upgrade de herramientas.	El servicio técnico.
<code>internal/verify/</code>	Chequeos de salud post-instalación + reportes.	La inspección final.
<code>internal/installcmd/</code>	Resuelve el comando correcto según tu SO (brew/apt/pacman/dnf/winget).	El traductor de idiomas de package managers.

📖 Docs y tests

CARPETA	QUÉ HACE
<code>docs/</code>	Toda la documentación (incluida la carpeta <code>docs/codebase/</code> para mantenedores).
<code>e2e/</code>	Tests end-to-end con Docker (Ubuntu + Arch).
<code>testdata/golden/</code>	"Fotos" de la salida esperada, para detectar cambios no deseados.

6. La memoria (Engram): dónde termina Gentle-AI y empieza otra cosa

Esto confunde a todos, así que ojo:

- **Gentle-AI** *instala y configura* Engram. Le dice al agente: "usá estas herramientas de memoria".
- **Engram** (proyecto externo) es el que *realmente guarda y busca* la memoria.

```
El agente recibe tu prompt
↓
El prompt (que puso Gentle-AI) le dice: "usá las herramientas de memoria"
↓
El agente llama a `engram mcp --tools=agent`
  ↓ guardar: mem_save / mem_session_summary
  ↓ buscar:  mem_context / mem_search / mem_get_observation
  ↓
Engram guarda/busca fuera del código de Gentle-AI
```

Traducción: Gentle-AI pone el enchufe; Engram es el electrodoméstico que consume luz. El código de la base de datos de memoria **NO** vive en este repo.

7. `install` vs `sync` : ¿cuál es la diferencia?

	GENTLE-AI INSTALL	GENTLE-AI SYNC
Cuándo	La primera vez.	Cada vez que querés refrescar la config.
Qué hace	Instala todo desde cero.	Actualiza solo lo que quedó viejo.
Regla clave	Hace backup antes.	Es idempotente : si ya está todo al día, no toca nada (<code>FilesChanged == 0</code>).

Idempotente = correrlo dos veces da el mismo resultado que correrlo una. No rompe nada.

8. Cosas que Gentle-AI NO es (para no confundirte)

- **✗** No es la base de datos de memoria (eso es Engram).
- **✗** No es un servidor de dashboard (no hay servidor HTTP en este repo).
- **✗** No es un package manager genérico (solo instala lo suyo).
- **✗** No es el agente de IA en sí (Claude, Cursor, etc. corren aparte).

9. Las 6 reglas sagradas (invariantes)

1. **Backup antes de tocar** — siempre hay rollback.
2. **Los adapters mandan en las rutas** — cada agente sabe sus caminos.
3. **Los components mandan en el contenido** — reutilizable entre agentes.
4. **El planner ordena** — no ordenes dependencias a mano en el CLI o TUI.
5. **El sync es idempotente** — correrlo dos veces no reescribe lo que ya está.
6. **Lo externo se queda afuera** — no documentes ni toques internals de Engram acá.

10. ¿Dónde toco si quiero cambiar X?

QUIERO...	EMPIEZO POR...
Agregar un agente soportado	<code>internal/model/types.go</code> + <code>internal/catalog/agents.go</code>
Cambiar la config de memoria	<code>internal/components/engram/</code>
Cambiar flags del CLI	<code>internal/cli/</code>
Cambiar una pantalla visual	<code>internal/tui/model.go</code> + <code>router.go</code>
Cambiar el orden de instalación	<code>internal/planner/</code> + <code>internal/pipeline/</code>
Cambiar backups/restore	<code>internal/backup/</code> + <code>internal/cli/restore.go</code>

Glosario rápido

- **CLI**: programa de terminal que corrés con comandos y flags.
- **TUI**: interfaz visual dentro de la terminal (con colores y navegación).
- **MCP**: protocolo para que el agente hable con herramientas externas.
- **SDD**: Spec-Driven Development. Pensar y especificar antes de codear.
- **Engram**: el sistema de memoria persistente (externo).
- **Adapter**: sabe DÓNDE va la config de un agente.
- **Component**: sabe QUÉ contenido inyectar.
- **Idempotente**: correrlo N veces = correrlo 1 vez.
- **Golden file**: foto de la salida esperada para tests.
- **Rollback**: volver atrás a un estado anterior con el backup.

Para el detalle técnico de mantenedores, ver `docs/CODEBASE-GUIDE.md` y `docs/codebase/`.